System Verification and Validation Plan for RoCam

Team #3, SpaceY
Zifan Si
Jianqing Liu
Mike Chen
Xiaotian Lou

March 5, 2026

Revision History

Date	Version	Notes
2025-10-21	1.0	Initial Draft
2025-10-23	2.0	Revised on section 3
2025-10-26	3.0	Revised on section 3 based on TA and peer feedback
2025-10-27	4.0	Added system tests
2026-03-04	5.0	Scope limited to unit testing and stakeholder (McMaster Rocketry Team) review only; removed supervisor, classmate, task-based inspection, field testing
2026-03-05	6.0	Added unit testing plan

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 In-Scope Objectives . . . 2
- 2.4 Out-of-Scope Objectives . . . 2
- 2.5 Extras . . . 2
- 2.6 Relevant Documentation . . . 3

3 Plan **3**
- 3.1 Verification and Validation Team . . . 4
- 3.2 SRS Verification . . . 4
  - 3.2.1 functional and Non-Functional Requirement Review . . . 4
  - 3.2.2 Stakeholder Review . . . 4
  - 3.2.3 Constraints and Scope of Work Review . . . 4
- 3.3 Design Verification . . . 5
- 3.4 Verification and Validation Plan Verification . . . 7
- 3.5 Implementation Verification . . . 8
- 3.6 Automated Testing and Verification Tools . . . 9
- 3.7 Software Validation . . . 10

4 System Tests **11**
- 4.1 Tests for Functional Requirements . . . 11
  - 4.1.1 Area of Testing: Video Acquisition and Output . . . 11
  - 4.1.2 Area of Testing: Recording . . . 13
  - 4.1.3 Area of Testing: Tracking . . . 14
  - 4.1.4 Field Testing (Out of Scope) . . . 16
- 4.2 Tests for Nonfunctional Requirements . . . 16
  - 4.2.1 UI Walkthrough . . . 16
  - 4.2.2 Code and Documentation Walkthrough . . . 19
  - 4.2.3 Error Handling and Logging . . . 19
  - 4.2.4 Field Testing (Out of Scope) . . . 20
- 4.3 Traceability Between Test Cases and Requirements . . . 22

5 Unit Test Description **23**
- 5.1 Unit Testing Scope . . . 23
- 5.2 Tests for Functional Requirements . . . 24
  - 5.2.1 Module: State Management (`mState`) . . . 24
  - 5.2.2 Module: API Gateway (`mAPI`) . . . 25
  - 5.2.3 Module: Recording Database (`mRecord`) . . . 26
  - 5.2.4 Module: Gimbal Abstraction (`mGimbal`) . . . 28
  - 5.2.5 Module: Tracking (`mTrack`) . . . 29

5.2.6 Module: CV Process Management (mCVManagement) . . . 30
5.2.7 Module: Live Video Process Management (mVideoManagement) . . . 31
5.2.8 Module: Transcode API (mAPI) . . . 32
5.2.9 Module: Transcode Process (mTranscode) . . . 32
5.2.10 Module: CV Process (mCV) . . . 33
5.2.11 Module: Common IPC (Shared Modules) . . . 34
5.2.12 Module: Common IPC Buffer . . . 35
5.2.13 Module: Common Watchdog . . . 35
5.2.14 Module: Preview (mPreview) . . . 36
5.2.15 Module: Manual Control (mManual) . . . 36
5.2.16 Module: Recording Management (mRecordMng) . . . 37
5.2.17 Module: Configuration (mConfig) . . . 38
5.2.18 Module: API Client (Network Layer) . . . 39
5.2.19 Module: Rocam Provider (React Context) . . . 40
5.2.20 Module: Utility Formatters . . . 41
5.2.21 Module: Internationalization (i18n) . . . 42
5.2.22 Module: Application Routing (App) . . . 42
5.3 Tests for Nonfunctional Requirements . . . 42
5.3.1 Module: State Management (mState) . . . 42
5.3.2 Module: API Gateway (mAPI) . . . 43
5.3.3 Module: SSE (Server-Sent Events) . . . 43
5.3.4 Module: IPC Buffer . . . 44
5.3.5 Module: CV Process Management (mCVManagement) . . . 44
5.3.6 Module: Recording Database (mRecord) . . . 45
5.3.7 Module: Watchdog . . . 45
5.3.8 Module: System Status (mStatus) . . . 45
5.3.9 Module: Common Utils . . . 46
5.3.10 Module: Recording-Only State Management . . . 46
5.3.11 Module: Main Entry Point . . . 47
5.3.12 Module: Utility Formatters . . . 48
5.3.13 Module: UI Components . . . 49
5.4 Traceability Between Test Cases and Modules . . . 50

6 Appendix **54**
6.1 User Survey Questions . . . 54

1 Symbols, Abbreviations, and Acronyms

symbol	description
Yolo	You only look once – an existing object detection model
CNN	Convolutional Neural Network
Jetson	abbreviatio for specifcally Jetson Nano Orin, which is the proposed board for this project
STM32	abbreviation for specifically STM32 microcontroller, which is the proposed microcontroller

This Verification and Validation (V&V) plan outlines how the RoCam project will demonstrate that the system is built correctly (verification) and fits stakeholder needs (validation). It anchors testing to the SRS, architecture (MG), and detailed design (MIS), ensuring every requirement is traceable where possible. **The only testing performed is unit testing;** integration, system, and field testing are out of scope. Verification relies on structured internal reviews and checklists, plus unit tests, to confirm functional behavior and safety assumptions. Validation is limited to a planned review with the McMaster Rocketry Team (stakeholders) to collect feedback; no task-based inspection, classmate review, or supervisor validation is conducted. Roles and responsibilities are assigned to team members only, with feedback from the rocketry team captured and tracked. Priorities emphasize correctness, performance, reliability, and safety; lower-priority or out-of-scope items (e.g., exhaustive usability studies, independent verification of third-party components, and all testing beyond unit tests) are noted to keep the plan feasible. The document concludes with traceability tables, placeholders for unit-level details after design finalization, and appendices for parameters and optional surveys—forming a clear, practical roadmap for the evidence of quality that can be gathered under these constraints.

2 General Information

2.1 Summary

The RoCam system is a ground-based camera and gimbal assembly designed to autonomously track model rockets during launch and flight. It utilizes a high-resolution camera coupled with a motorized gimbal to maintain visual lock on the rocket, providing real-time data and video feed. The system consists of a STM32 based motion controller that interfaces with the physical gimbal; a Jetson compute module that processes the camera feed; and a react frontend that displays the video preview and allows the user to control the gimbal. The primary purpose of RoCam is to replace the traditional unreliable manual tracking methods with an automated solution to produce high-quality flight data and video footage. To better assist model rocket teams across Canada to conduct successful launches by providing an affordable and reliable system that can be easily track launching movements.

2.2 Objectives

The primary objective of this Verification and Validation V&V Plan is to establish confidence in the correctness, performance, and reliability of the RoCam system by ensuring that all verification and validation activities are thorough, traceable, and aligned with project requirements. The plan emphasizes unit testing and internal review to confirm that functional and performance criteria are addressed where possible, with improvements tracked through GitHub Issues. Validation focuses on a planned review with the McMaster Rocketry Team (stakeholders) to collect feedback; no field testing or supervisor validation is performed. These qualities—correctness, performance, reliability, and traceability—are fundamental to the success of the V&V process, ensuring that the RoCam system is both technically sound and operationally dependable. Certain objectives, such as independent verification of third-

party hardware and extended usability testing, are excluded due to scope and resource limitations, with reliance placed on manufacturer and stakeholder validation.

2.3 In-Scope Objectives

Verification and validation efforts will prioritize the following qualities:

- **Correctness:** Ensure the reliability of tracking and control algorithms through testing and validation.
- **Performance:** Achieve real-time processing of 1080p 60 fps video and maintain minimal gimbal control latency.
- **Safety:** Guarantee that the system operates safely under assumed conditions, with fail-safe mechanisms.

2.4 Out-of-Scope Objectives

The following objectives are excluded from this project due to time, budget, and resource constraints:

- **Third-Party Component Verification:** External hardware such as the gimbal controller and camera are assumed to perform according to their manufacturer specifications.
- **Scalability Beyond Small-Scale Rockets:** Tracking a bullet or extremely difficult targets beyond a model rocket is outside the scope of this project.

By defining these priorities and exclusions, the RoCam project ensures that available effort is directed toward validating the most critical system qualities: **performance, accuracy, and operational safety**.

2.5 Extras

Extras

Beyond the core objectives, the RoCam project includes several extras that enhance its quality and usability:

- **STM32 control circuit design:** Development of a custom control circuit to interface the gimbal actuators with the host computer, enabling precise motion control and feedback sensing.
- **User Instructional Video:** Creation of a short instructional video demonstrating system setup, calibration, and operation to support new users and ensure safe deployment.

These extras aim to improve the overall usability, accessibility, and documentation quality of the final deliverable, reinforcing RoCam's emphasis on practical integration between computer vision software, mechanical control, and end-user experience.

2.6 Relevant Documentation

Software Requirements Specification (SRS Vision-Guided Tracker Team (2025c)): The SRS document serves as a outline for the VnV plan. The SRS document contains specific functional and non-functional requirements, performance criteria, and constraints. The SRS document also contains detailed stakeholder roles and scope of the work. Aligning the VnV plan with the SRS document ensures that all specified requirements are adequately verified and validated.

Software Architecture MG (MG Vision-Guided Tracker Team (2025a)) The MG document provides details of our system architecture. To effectively design tests to validate and verify the system, it is important to understand the architecture and functionality of each module.

Software Detailed Design Document (MIS Vision-Guided Tracker Team (2025b)): The MIS document provides detailed design of our system. It provides critical information about the implementation details, including data structures, algorithms, and interfaces. This document is essential for efficient test design to specific modules.

3 Plan

This section outlines the overall Verification and Validation (V&V) plan for the **RoCam** project. It defines the structure, responsibilities, and methods used to verify that the system meets its functional and non-functional requirements and to validate that it fulfills stakeholder needs. The section covers the verification of the SRS, design, and implementation via internal reviews and unit testing only, as well as the validation strategy through a planned stakeholder review with the McMaster Rocketry Team (no field testing). Its purpose is to provide a clear, organized framework ensuring that all aspects of RoCam are systematically tested, reviewed, and proven reliable before final deployment.

3.1 Verification and Validation Team

Table 1: Verification and Validation Team Responsibilities

Name	Role	V&V Responsibilities
Zifan Si	Front End Interface Tester	Responsible for verifying and validating the usability and functionality of the react frontend interface. Ensure usability, robustness of frontend features.
Jianqing Liu	Team Lead	Conduct integration test and system validation. Validate the functionality and performance of the system against requirements and use cases. Validate system design and functionality.
Mike Chen	CV Tester	Verify and validate computer vision pipeline performance, including object detection and tracking accuracy.
Xiaotian Lou	Integration Tester and test automation developer	Implement automated testing frameworks in all software components. Implement API, and integration test across different modules.

3.2 SRS Verification

The SRS verification will focus on verify whether the system design satisfy all functional and non-functional requirements, stakeholder needs, scope of work and constraints specified. The SRS verification will be conducted through the following steps:

3.2.1 functional and Non-Functional Requirement Review

Since the Design function will evaluate in detail whether the system align with the functional and non-functional requirements. The SRS verification will focus on verify the system design as a whole against the requirements. It will be implemented as integration test in the final stages of development.

3.2.2 Stakeholder Review

Since our team leader is the control team lead of the McMaster Rocketry Team, He will be the one to review the system design and implementation and the communication bridge between the team and the stakeholders.

3.2.3 Constraints and Scope of Work Review

Several internal review sessions will be conducted to ensure that our system design and implementation are within the defined scope of work and constraints specified in the SRS document.

Table 2: SRS Verification Checklist

Phase	**Verification Activities**
Functional & Non-Functional Requirement Review	☐ Review all requirements against current system design ☐ Create automated system tests for functional requirements ☐ Measure performance, safety, and reliability against non-functional requirements ☐ Record test results and address any failed cases ☐ Update traceability matrix after verification
Stakeholder Review	☐ Present verified results and collect feedback ☐ Document required changes based on feedback ☐ Obtain stakeholder confirmation or approval
Constraints & Scope of Work Review	☐ Verify implementation stays within project scope ☐ Check compliance with all technical and safety constraints ☐ Review design against timeline and budget limits ☐ Update records if scope or constraints change
Integration Verification	☐ Conduct integration tests covering all system components ☐ Validate full system behavior against SRS requirements ☐ Log and resolve all integration issues ☐ Prepare final verification report for stakeholder review
Safety Review	☐ Review safety requirements against Hazard Analysis

3.3 Design Verification

The design verification process for the **RoCam** project ensures that the proposed architecture, algorithms, and hardware interfaces satisfy all functional and non-functional requirements before implementation. Design verification activities will be conducted through structured design reviews, peer evaluations, and the use of detailed verification checklists. The goal is to confirm that the design meets the intended performance, reliability, and safety objectives defined in the SRS.

Verification Plan

The design verification will proceed through internal review only (no classmate or supervisor review, due to contact and resource limitations):

1. **Internal Design Review:** Each subsystem (computer vision, control, and UI) will undergo an internal review by each team member. Each member must agree on the designed architecture, algorithms, and interfaces. Interfaces must be clearly defined for scalability.

Verification Artifacts

Design verification will produce the following artifacts:

- Verified design diagrams and interface specifications.
- Internal review notes and agreed-upon design decisions.
- Updated design documents reflecting corrections and improvements.
- Completed design verification checklists (see below).

Design Verification Checklist

The following checklist will be used during design reviews to ensure completeness and consistency across all modules:

Table 3: Design Verification Checklist

Phase	**Verification Activities**
Internal Design Review	☐ Review subsystem architectures (vision, control, UI) with all team members ☐ Confirm APIs that are agreed upon ☐ Verify all module interfaces are clearly defined and documented in code ☐ Ensure design supports scalability and modularity ☐ Record review comments and update design diagrams
Verification Artifacts	☐ Store verified design diagrams and updated interface specifications ☐ Archive internal review notes and design decisions ☐ Maintain a design document in the project repository ☐ Complete and file the final design verification checklist

By following this structured verification plan and checklist, the team ensures that the RoCam design is both technically sound and fully aligned with its performance and safety goals prior to implementation.

3.4 Verification and Validation Plan Verification

The verification of the verification and validation (VnV) plan for the **RoCam** project will be verified through internal review and unit testing only. Each iteration of feedback and improvements will be tracked with GitHub Issues. The focus of the verification plan is to ensure functionality and performance metrics are adequately covered in design where possible.

For the validation plan, the only validation activity is a planned review with the McMaster Rocketry Team (stakeholders) to collect feedback. No field tests, integration tests, or supervisor validation are performed; the only testing conducted is unit testing.

Table 4: RoCam Verification and Validation (V&V) Checklist

Phase	**Actionable Verification and Validation Activities**
Verification Plan Review	☐ Conduct internal reviews of the V&V plan with team ☐ Ensure functionality and performance metrics are fully covered ☐ Track feedback and revisions using GitHub Issues ☐ Confirm all review comments are resolved before approval
Automated Verification Testing	☐ Develop automated tests for vision, control, and UI components ☐ Verify software behavior under simulated conditions ☐ Record test results and compare against performance criteria ☐ Update test scripts based on identified defects or regressions
Continuous Monitoring	☐ Review unit test outcomes after each iteration ☐ Maintain test evidence and validation summaries in repository ☐ Update the V&V plan to reflect system improvements and new unit test cases

3.5 Implementation Verification

The implementation verification for the **RoCam** project ensures that the developed software and integrated hardware components correctly realize the verified design. This phase confirms that the implemented system satisfies the functional and non-functional requirements defined in the SRS through a combination of structured testing, static verification, and peer review.

Dynamic Verification

Due to resource and contact limitations, **only unit testing** is performed. Integration testing, system testing, and field testing are out of scope.

- **Unit Testing:** Each software module (vision processing, gimbal control, data recording, and UI management) will undergo unit testing to verify functional correctness in isolation. This is the only form of dynamic testing conducted.

Out of scope: Integration testing, system testing, and field testing are not performed due to the constraints noted in this plan.

Static Verification

Static verification will be performed throughout the implementation phase to ensure that the codebase meets quality, safety, nd maintainability standards before execution. Planned techniques include:

- **Code Walkthroughs:** Internal code walkthroughs where each subsystem lead presents core implementation logic and design decisions to the team, enabling identification of potential design or coding issues.
- **Code Inspection:** Team members will review each other's pull requests prior to merging into the main branch. Reviews will focus on coding standard compliance, potential logic errors, exception handling, and safety-critical sections align with the Hazard Analysis (e.g., gimbal actuation and state transitions).
- **Static Analysis Tools:** Where applicable (e.g., on the frontend), static analysis may be used to detect common implementation defects. Results will be logged and reviewed as part of the verification record.
- **Documentation Review:** The team will verify that all inline comments, module-level documentation, and API references are consistent with the design specification and user documentation requirements.

Together, these verification activities ensure that the RoCam implementation is thoroughly tested, statically verified, and validated against both its design and real-world performance expectations.

3.6 Automated Testing and Verification Tools

The RoCam project adopts a structured automated testing approach that prioritizes maintainability, integration compatibility, and developer productivity. The selection of testing frameworks was guided by three key criteria: (1) **language and runtime fit**, ensuring each tool aligns with its respective language and runtime; (2) **ease of automation**, to integrate smoothly with CI/CD pipelines; and (3) **scalability**, so that test coverage can expand as the project evolves. These principles ensure a consistent and reliable testing environment across frontend and end-to-end validation stages.

Each subsystem of RoCam employs unit testing and coverage tools appropriate to its implementation language. The frontend (React/TypeScript) uses a unit testing framework and coverage reporting suited to that stack. System-level and Web UI testing, where in scope, use tooling that can coordinate frontend and hardware layers. Specific frameworks are chosen to support CI/CD integration and to allow test coverage to scale as the project evolves.

Code Coverage and Reporting Plan:
Coverage analysis is conducted using tools that provide line, branch, and function-level metrics for each language stack, with results suitable for unified tracking and CI integration.

Linters and Formatters:
To maintain coding standards and prevent stylistic or logical errors, RoCam enforces consistent linting and formatting on the frontend:

- **Frontend (TypeScript):** Uses **ESLint** and **Prettier** to maintain uniform style rules and detect syntax or logical inconsistencies.

3.7 Software Validation

Software validation confirms that **RoCam** satisfies stakeholder needs and accurately implements the intended use cases ("building the right system"). Validation complements verification and is referenced to the SRS goals and stakeholder roles.

Stakeholder Review (McMaster Rocketry Team)

Validation is limited to a planned review with the McMaster Rocketry Team (stakeholders). The following is **in scope**:

- **Stakeholder Review Session:** A scheduled review with the McMaster Rocketry Team to present the current system (or design/SRS), discuss scope, priorities, and acceptance criteria, and collect feedback. Revisions to the plan or implementation will be made based on their feedback.

Out of scope (not conducted): Task-based inspection by stakeholders, classmate or peer review, supervisor validation, Rev 0/supervisor demo, and formal user (operator) sessions. These are excluded due to contact and resource limitations.

External Data and Bench Validation

- **Reference Footage:** Where live launches are unavailable, public model-rocket videos and internally recorded dry-runs are used to validate detection/tracking logic and reacquisition behavior.
- **Synthetic Scenarios:** Scripted clips with occlusion/smoke/backlight and varying apparent rocket sizes validate robustness claims prior to field tests.

Acceptance and Traceability

- **Acceptance Criteria:** Derived from the SRS (e.g., sustained 1080p 60 fps I/O, end-to-end latency ≤ 120 ms, successful tracking through nominal flight phases, recording with a ngle overlays).
- **Traceability:** Validation tests map to SRS goals and stakeholder needs; a table links each validation activity to the corresponding SRS item(s) and planned evidence (videos, logs, checklists).

If no suitable external datasets are available for certain scenarios, this limitation will be stated explicitly; validation in this plan relies on the planned stakeholder review with the McMaster Rocketry Team and, where applicable, unit testing and internal review only. This section also references the SRS verification section for consistency checks between validated behaviors and specified requirements.

4 System Tests

Scope note: Due to resource and contact limitations, the only testing actually performed in this project is **unit testing**. The system tests and field tests described in this section are documented for reference and traceability but are **out of scope** and will not be executed. Evidence of correctness will come from unit tests and from the planned stakeholder review with the McMaster Rocketry Team.

4.1 Tests for Functional Requirements

The following manual tests are designed to evaluate the robustness of the Tracking Camera System. Each test includes its initial state, input, expected output, justification (test case derivation), and procedure. Tests are organized by functional area. The intent is to verify compliance with the functional requirements (FR) while deliberately stressing the system to expose edge cases.

4.1.1 Area of Testing: Video Acquisition and Output

test-aquire-frame: Camera Frames are Acquired at Startup

- **Control:** Manual
- **Initial State:** System powered off; camera connected; HDMI monitor attached.
- **Input:** Power on the system and allow it to fully initialize.
- **Expected Output:** Within 10s of boot, logs show that the camera is acquiring frames.
- **Test Case Derivation:** The system's operation depends on a valid video feed.
- **How Test Will Be Performed:** Boot the device; observe the program logs.

test-disconnect-camera: Camera Disconnect During Operation

- **Control:** Manual
- **Initial State:** System initialized; preview and HDMI output active.
- **Input:** Physically disconnect the camera from the system.
- **Expected Output:** On disconnect, the UI and HDMI output show a clear error message; no crash.
- **Test Case Derivation:** When the camera is disconnected, the system should gracefully handle the loss without crashing. And it should notify the operator that the camera is disconnected.
- **How Test Will Be Performed:** Disconnect the camera from the system; observe the UI and HDMI output for error messages.

test-hdmi-output: HDMI Output with Info Overlay

- **Control:** Manual
- **Initial State:** System initialized; idle state; preview and HDMI output active.
- **Input:** Move the gimbal manually through the UI.
- **Expected Output:** HDMI shows 1080p60 camera feed with a text overlay of current pan/tilt angles.
- **Test Case Derivation:** The system should be able to display the current camera feed and the pan/tilt angles for the viewers
- **How Test Will Be Performed:** Move the gimbal manually, and visually verify overlay values change smoothly and match commanded motion

test-disconnect-hdmi: HDMI Disconnect

- **Control:** Manual
- **Initial State:** System initialized; preview and HDMI output active.
- **Input:** Physically disconnect the HDMI cable.
- **Expected Output:** On disconnect, the UI show a clear error message; no crash.
- **Test Case Derivation:** When the HDMI cable is disconnected, the system should gracefully handle the loss without crashing. And it should notify the operator that the HDMI cable is disconnected.
- **How Test Will Be Performed:** Disconnect the HDMI cable from the system; observe the UI for error messages.

test-preview-real-time: Preview with Real-Time Info

- **Control:** Manual
- **Initial State:** System initialized; idle state; preview and HDMI output active.
- **Input:** Move the gimbal manually through the UI.
- **Expected Output:** Preview in the UI updates fluidly (target $\geq$15fps) and displays current pan/tilt angles.
- **Test Case Derivation:** The system should allow the camera operator to see the current pan/tilt angles and HDMI output in real-time.
- **How Test Will Be Performed:** Move the gimbal manually, and visually verify the preview in the UI updates fluidly and displays the current pan/tilt angles.

test-network-disconnect: Network Disconnect While Viewing UI

- **Control:** Manual
- **Initial State:** System initialized; preview and HDMI output active.
- **Input:** Disconnect the system from the network for 30s; reconnect.
- **Expected Output:** The system keeps functioning as normal without network; UI show a clear error message; upon reconnection, the UI session restores automatically within 5s without reboot; no data loss or crash.
- **Test Case Derivation:** The system need to be resilient against transient network failures in field conditions.
- **How Test Will Be Performed:** Manually disconnect and connect the system from the network; observe the UI for error messages.

4.1.2 Area of Testing: Recording

test-recording: Start and Stop Recording

- **Control:** Manual
- **Initial State:** System initialized; idle state; sufficient disk space.
- **Input:** Press "Start Recording", wait 10s, then press "Stop Recording".
- **Expected Output:** A 10 second 1080p60 video file is created and saved to the disk.
- **Test Case Derivation:** The system should be able to start and stop recording.
- **How Test Will Be Performed:** Press "Start Recording", wait 10s, then press "Stop Recording".

test-disk-full: Disk Full Behavior During Recording

- **Control:** Manual
- **Initial State:** System initialized; idle state; storage nearly full (simulate by filling disk leaving $<$ 1min recording capacity).
- **Input:** Start recording and allow disk to fill.
- **Expected Output:** UI show a clear "Storage Full" alert; current recording is stopped and the file is saved to the disk; no crash.
- **Test Case Derivation:** The system should be able to handle storage exhaustion gracefully.
- **How Test Will Be Performed:** Fill the disk with data until it is nearly full; start recording; observe the UI for the "Storage Full" alert; check the disk for the saved file.

test-manage-recordings: List and Download Recordings

- **Control:** Manual
- **Initial State:** At least two prior recordings exist.
- **Input:** Open recordings list; download the latest file and an older file; delete a small test clip.
- **Expected Output:** List shows correct entries (name, timestamp, duration); downloads succeed; delete removes the selected item only.
- **Test Case Derivation:** The system should be able to list, delete, and download recordings.
- **How Test Will Be Performed:** Open the recordings list; download the latest file and an older file; delete a small test clip; check the list for the correct entries; check the disk for the saved files.

4.1.3 Area of Testing: Tracking

test-manual-control: Manual Gimbal Control in Idle

- **Control:** Manual
- **Initial State:** System initialized; idle state; preview and HDMI output active.
- **Input:** Issue manual pan and tilt commands via UI.
- **Expected Output:** The gimbal follows commands smoothly.
- **Test Case Derivation:** The operator should be able to manually control the gimbal in idle state.
- **How Test Will Be Performed:** Issue manual pan and tilt commands via UI; observe the gimbal movement.

test-transition-to-tracking: Transition to Tracking When Rocket Detected

- **Control:** Manual
- **Initial State:** System initialized; idle state; preview and HDMI output active.
- **Input:** Arm the system via UI, and waving a model rocket in front of the camera.
- **Expected Output:** The system transitions to tracking state as soon as the rocket is detected; the UI shows the system is in tracking state
- **Test Case Derivation:** The system should be able to transition to tracking state when a rocket is detected.
- **How Test Will Be Performed:** Arm the system via UI, and waving a model rocket in front of the camera; observe the UI for the system to transition to tracking state.

test-tracking: Keep Rocket in Frame While Tracking

- **Control:** Manual
- **Initial State:** System initialized; tracking state; preview and HDMI output active.
- **Input:** Continue waving the model rocket in front of the camera.
- **Expected Output:** Gimbal should continuously adjust its position to keep the rocket in the frame for the duration of the test.
- **Test Case Derivation:** The system should be able to keep the rocket in the frame while tracking.
- **How Test Will Be Performed:** Wave the model rocket in front of the camera; observe the gimbal movement; observe the preview in the UI.

test-return-to-idle: Return to Idle When Rocket is Lost

- **Control:** Manual
- **Initial State:** System initialized; tracking state; preview and HDMI output active.
- **Input:** Occlude the model rocket.
- **Expected Output:** Within 2s of sustained loss, system transitions to idle state; ceases autonomous gimbal commands; UI indicates "target lost".
- **Test Case Derivation:** The system should be able to return to idle state when the rocket is lost.
- **How Test Will Be Performed:** Occlude the model rocket; observe the UI for the system to transition to idle state.

test-exit-gimbal-range: Rocket Exits Gimbal Range

- **Control:** Manual
- **Initial State:** System initialized; tracking state; preview and HDMI output active.
- **Input:** Move the model rocket outside of the gimbal's range of motion.
- **Expected Output:** The gimbal moves until it reaches its mechanical limit, then stops moving further; no errors or warnings are presented to the user; system remains responsive.
- **Test Case Derivation:** The system must safely handle objects that leave the gimbal's range without faulting or reporting spurious errors.
- **How Test Will Be Performed:** Move the model rocket outside of the gimbal's range of motion; observe the gimbal reaches its mechanical end and stops; verify that no error messages appear in the UI and that all other system functions remain operational.

4.1.4 Field Testing (Out of Scope)

Field testing is **out of scope** for this project due to resource and contact limitations. The following is retained for reference only.

If field testing were to be conducted, it would be done by the McMaster Rocketry Team (client), who would be provided with the manual and would operate the system at a model rocket launch event. The development team would be present to assist. After the event, the client could fill out a survey (Appendix 6.2). Under the current plan, no field testing is performed; only unit testing and a planned stakeholder review with the rocketry team are conducted.

4.2 Tests for Nonfunctional Requirements

4.2.1 UI Walkthrough

The UI walkthrough will be conducted by the capstone team members themselves before the field test.

test-consistent-units: Consistent and Changeable Units

- **Type:** Manual
- **Initial State:** System initialized; idle state; preview and HDMI output active.
- **Input/Condition:** Go through the UI twice, once with metric units, once with imperial units.
- **Output/Result:** The units are consistent and changeable.
- **How Test Will Be Performed:** The capstone team members will go through the UI twice, once with metric units, once with imperial units; observe the units in the UI.

test-bilingual-ui: Bilingual UI

- **Type:** Manual
- **Initial State:** System initialized; language = English.
- **Input/Condition:** Switch UI language to French and back; navigate all UI sections, dialogs, and toasts.
- **Output/Result:** All visible strings/localized assets appear in the selected language; no truncation/overflow; no mixed-language strings; hotkeys remain functional; date/time/number formats localize correctly.
- **How Test Will Be Performed:** Perform a complete UI walkthrough in French; capture screenshots of each page/dialog; switch back to English and repeat spot checks.

test-immediate-ui-feedback: Immediate UI Feedback

- **Type:** Manual
- **Initial State:** System idle; UI ready.
- **Input/Condition:** Trigger representative actions: button press, toggle, menu selection, starting/stopping recording, arming/disarming, opening settings.
- **Output/Result:** A visible confirmation (pressed state, spinner, toast, label change) appears within 0.2s for each action.
- **How Test Will Be Performed:** Observe the UI while performing each action

test-visual-indicators: Rely on Visual Indicators

- **Type:** Manual
- **Initial State:** System initialized; speakers muted or disconnected.
- **Input/Condition:** Trigger statuses/alerts: armed, tracking, target lost, storage low/full, network disconnect, device error.
- **Output/Result:** Every status/alert is presented visually (icons, colors, banners, toasts) without relying on sound; no action-critical info is audio-only.
- **How Test Will Be Performed:** With audio disabled, induce each state/fault; verify a visible indicator appears and remains long enough to be noticed.

test-confirmation-prompts: Confirmation Prompts for Safety-Critical Actions

- **Type:** Manual
- **Initial State:** System idle; gimbal powered.
- **Input/Condition:** Attempt potentially hazardous actions: force home while tracking, hard stop, high-speed slew, firmware update during armed state, factory reset.
- **Output/Result:** A blocking confirmation dialog appears for each action; dialog text states the specific risk and outcome; default action is "Cancel"; action proceeds only after explicit confirmation.
- **How Test Will Be Performed:** Trigger each action; capture screenshots of the prompt; verify risk wording is specific (not generic); verify no motion/change occurs until "Confirm" is pressed.

test-manual-idle-mode: Manual Idle Mode / Override

- **Type:** Manual
- **Initial State:** System armed or tracking.
- **Input/Condition:** Press "Idle" (or equivalent) control while tracking.
- **Output/Result:** System transitions to idle within 1 s; autonomous commands cease; manual controls become available; status clearly shows "Idle."
- **How Test Will Be Performed:** Enter tracking, then invoke manual idle; observe immediate cessation of gimbal motion; verify state banner change and manual jog controls responsiveness.

test-no-collection-pii: No Collection of PII

- **Type:** Manual
- **Initial State:** System initialized; default configuration.
- **Input/Condition:** Inspect all settings, logs, exported files, filenames/paths, and network-configurable fields.
- **Output/Result:** No fields request or store personal identifiers (names, emails, phone numbers, precise home addresses, faces with identity tags); logs contain only technical data; telemetry excludes PII.
- **How Test Will Be Performed:** Open each settings and log panel; download logs/recordings; inspect metadata (EXIF/JSON/CSV) for PII keys; verify any optional labels are device/session IDs, not personal data.

test-color-vision-deficiency: Color-Vision Deficiency Accessibility

- **Type:** Manual
- **Initial State:** System initialized; all status indicators visible.
- **Input/Condition:** Review all status/alert states (normal, armed, tracking, warning, error) and interactive controls (enabled/disabled) under simulated Protanopia/Deuteranopia/Tritanop or printed CVD reference swatches.
- **Output/Result:** Each state is distinguishable without relying on hue alone (uses shape, text labels, patterns, or contrast); contrast ratio $\geq$ 4.5:1 for text/icons against background.
- **How Test Will Be Performed:** Cycle UI through all states; verify distinct icon shapes/labels; optionally view via a CVD simulator; record pass/fail with screenshots; confirm tooltips/textual labels exist for each color state.

4.2.2 Code and Documentation Walkthrough

test-manual-arm-mode: Manual Arm Mode Only

- **Type:** Static
- **Initial State:** N/A
- **Input/Condition:** Inspect all state transition code and configuration defaults.
- **Output/Result:** The system only enters armed mode when explicitly commanded by the user, never automatically.
- **How Test Will Be Performed:** Review code for all references to the armed state, verify guards require user input.

test-gimbal-interface: Gimbal Interface Documentation Review

- **Type:** Static
- **Initial State:** N/A
- **Input/Condition:** Review the interface specification with the client, including command set, telemetry, timing, and safety notes.
- **Output/Result:** The client confirms the interface documentation is complete, clear, and suitable for supporting multiple gimbal systems.
- **How Test Will Be Performed:** Conduct a walkthrough meeting with the client, check all required topics against a prepared checklist.

4.2.3 Error Handling and Logging

test-report-errors: Report All Errors to the User

- **Type:** Manual
- **Initial State:** System initialized and idle.
- **Input/Condition:** Manually trigger hardware, communication, and software faults.
- **Output/Result:** Each error generates a visible and descriptive message on the user interface.
- **How Test Will Be Performed:** Disconnect devices, kill processes, and simulate faults while observing the UI for clear error reporting.

test-stop-immediately: Stop Immediately on Unrecoverable Error

- **Type:** Manual
- **Initial State:** System operating in tracking mode.
- **Input/Condition:** Introduce a fatal condition such as gimbal communication loss or corrupted control process.
- **Output/Result:** The system immediately stops all automated operations and transitions to a safe idle state.
- **How Test Will Be Performed:** Disconnect the gimbal or crash the tracking process, verify that motion stops and the system displays a clear unrecoverable error message.

test-validate-commands: Validate All Commands

- **Type:** Manual
- **Initial State:** System idle.
- **Input/Condition:** Enter invalid or out-of-range user commands through the UI or API.
- **Output/Result:** The system rejects invalid commands and displays a validation error without sending them to the gimbal.
- **How Test Will Be Performed:** Attempt to send commands beyond gimbal limits or in unsupported formats and confirm that they are blocked with clear feedback.

test-log-operations: Log All Operations

- **Type:** Manual
- **Initial State:** System initialized and user logged in.
- **Input/Condition:** Perform a series of user operations such as arming, tracking, and recording.
- **Output/Result:** Each action is logged with a timestamp and operation details in the system log.
- **How Test Will Be Performed:** After the field test, review the log file to confirm all actions and timestamps are recorded accurately.

4.2.4 Field Testing (Out of Scope)

Field testing and user surveys are out of scope; only unit testing and the stakeholder review are performed.

test-user-survey: User Survey (Out of Scope)

- **Type:** Static (out of scope)
- **Initial State:** N/A
- **Input/Condition:** N/A
- **Output/Result:** Completed user survey.
- **How Test Will Be Performed:** Not performed. If field testing were in scope, the client would fill out the survey after the field test. Under the current plan, only unit testing and stakeholder review are conducted.

4.3 Traceability Between Test Cases and Requirements

Test ID	Requirements
test-aquire-frame	FR-1, SLR-2
test-disconnect-camera	FR-1
test-hdmi-output	FR-8, FR-9, INT-1, SLR-3
test-disconnect-hdmi	FR-8
test-preview-real-time	FR-10
test-network-disconnect	FR-10
test-recording	FR-11
test-disk-full	FR-11
test-manage-recordings	FR-12
test-manual-control	FR-2, FR-6
test-transition-to-tracking	FR-3, FR-4
test-tracking	FR-2, FR-3, FR-7
test-return-to-idle	FR-5
test-exit-gimbal-range	FR-2, FR-7
test-consistent-units	EZ-1, PI-1
test-bilingual-ui	PI-2
test-immediate-ui-feedback	EZ-2
test-visual-indicators	EZ-3
test-confirmation-prompts	EZ-4, UPR-3
test-manual-idle-mode	SCR-1
test-no-collection-pii	LR-1
test-color-vision-deficiency	AR-1
test-manual-arm-mode	SCR-4
test-gimbal-interface	INT-2
test-report-errors	RFR-1
test-stop-immediately	RFR-2
test-validate-commands	IR-1
test-log-operations	AUR-1
test-user-survey	AR-1, AR-2, SR-1, LR-1, UPR-1, UPR-2, PAR-1, SLR-1, RFR-5, CR-1, CR-2, SCR-2, SCR-3, RFR-3, EPE-1

5 Unit Test Description

5.1 Unit Testing Scope

The RoCam system unit testing covers all leaf modules implemented by the project team. The following modules are within the scope of unit testing:

- **Backend (Jetson) Modules:**
  - `mState` (State Management Module)
  - `mAPI` (API Gateway Module)
  - `mRecord` (Recording Database Module)
  - `mGimbal` (Gimbal Abstraction Module)
  - `mTrack` (Tracking Module)
  - `mCVManagement` (CV Process Management Module)
  - `mVideoManagement` (Live Video Process Management Module)
  - `mTranscodeManagement` (Transcode Process Management Module)
  - `mCV` (CV Process Module)
  - `mVideo` (Live Video Process Module)
  - `mTranscode` (Transcode Process Module)
  - Common utilities (IPC, IPC Buffer, Watchdog, System Status, Utils)
- **Frontend (UI) Modules:**
  - `mPreview` (Preview Module)
  - `mManual` (Manual Control Module)
  - `mRecordMng` (Recording Management Module)
  - `mConfig` (Configuration Module)
  - API client and network layer
  - Utility formatters and internationalization

Out of Scope:

- `mStream` (Video Stream Abstraction Module) — Implemented by Nvidia Deepstream SDK (third-party)
- `mSerial` (Serial Abstraction Module) — Implemented by pyserial library (third-party)
- `mStatus` (System Status Module) — Hardware-dependent library (jetson-stats)
- Hardware-specific integration tests (require actual Jetson hardware)

The unit testing strategy employs pytest for Python backend modules and Vitest for TypeScript frontend modules. Tests are designed to verify functional correctness, edge case handling, and error conditions for each module in isolation, with external dependencies mocked appropriately.

5.2 Tests for Functional Requirements

Unit tests for functional requirements verify that each module correctly implements its specified behavior. Tests are organized by module, with references to the MIS and MG documents where applicable. All tests are automated and implemented using pytest (Python backend) or Vitest (TypeScript frontend).

5.2.1 Module: State Management (`mState`)

The State Management module handles system state transitions, manual gimbal control, recording operations, and status aggregation. Tests verify state machine behavior, command validation, and data management.

1. test-state-management-arm-disarm

   Type: Functional, Dynamic, Automatic

   Initial State: StateManagement instance with mocked dependencies

   Input: Call arm() and disarm() methods

   Output: Internal armed flag set appropriately; gimbal LED commands issued

   Test Case Derivation: Verifies FR-4 (arming/disarming functionality) and that state transitions trigger correct hardware responses

   How test will be performed: Automated via test_state_management_class.py::TestArmDisarm

2. test-state-management-manual-move

   Type: Functional, Dynamic, Automatic

   Initial State: Disarmed system with gimbal at known position

   Input: manual_move() with directions up/down/left/right

   Output: Gimbal set_deg() called with correctly adjusted angles

   Test Case Derivation: Verifies FR-2 (manual control) and FR-6 (manual control only when disarmed)

   How test will be performed: Automated via test_state_management_class.py::TestManualMove

3. test-state-management-manual-move-clamped

   Type: Functional, Dynamic, Automatic

   Initial State: Disarmed system with gimbal near limits

   Input: manual_move() commands that would exceed gimbal limits

   Output: Angles clamped to valid ranges (tilt 0-90°, pan -45° to 45°)

   Test Case Derivation: Verifies IR-1 (command validation and clamping)

   How test will be performed: Automated via test_state_management_class.py::TestManualMove

4. test-state-management-recording

   Type: Functional, Dynamic, Automatic

   Initial State: System with mocked database and CV process

   Input: start_recording() and stop_recording() calls

   Output: Recording allocated in database; CV process notified; recording ID tracked

   Test Case Derivation: Verifies FR-11 (recording start/stop functionality)

   How test will be performed: Automated via test_state_management_class.py::TestRecording

5. test-state-management-download-pause

   Type: Functional, Dynamic, Automatic

   Initial State: Active recording session

   Input: on_download_start() and on_download_end() calls during recording

   Output: CV pipeline paused on first download, resumed after last download ends

   Test Case Derivation: Verifies correct pipeline management during recording downloads

   How test will be performed: Automated via test_state_management_class.py::TestDownloadHandlers

6. test-bounding-box-collection

   Type: Functional, Dynamic, Automatic

   Initial State: Empty BoundingBoxCollection

   Input: Sequential CVData with bounding boxes and timestamps

   Output: Correct bbox retrieval within 40ms window; latest valid bbox returned; max 10 entries enforced

   Test Case Derivation: Verifies data structure behavior for tracking data management

   How test will be performed: Automated via test_state_management.py

5.2.2 Module: API Gateway (`mAPI`)

The API Gateway module provides REST endpoints for UI communication. Tests verify endpoint behavior, request validation, and proper delegation to StateManagement.

1. test-api-endpoints-exist

   Type: Functional, Dynamic, Automatic

   Initial State: Flask test client with mocked StateManagement

Input: HTTP requests to /api/arm, /api/disarm, /api/manual_move, /api/manual_move_to, /api/recordings/start, /api/recordings/stop

Output: HTTP 200 responses; correct methods called on StateManagement

Test Case Derivation: Verifies FR-6, FR-10, FR-11 (API endpoints for control and recording)

How test will be performed: Automated via test_api.py and test_api_gateway.py

2. test-api-recordings-crud

   Type: Functional, Dynamic, Automatic

   Initial State: Flask test client with mocked database

   Input: GET /api/recordings, PATCH /api/recordings/¡id¿, DELETE /api/recordings/¡id¿

   Output: Recording list returned; rename and delete operations delegated to database

   Test Case Derivation: Verifies FR-12 (recording management)

   How test will be performed: Automated via test_api.py::TestRecordingsList, TestRecordingsRename, TestRecordingsDelete

3. test-api-validation

   Type: Functional, Dynamic, Automatic

   Initial State: Flask test client

   Input: PATCH /api/recordings/¡id¿ with missing or invalid new_name

   Output: HTTP 400 Bad Request with appropriate error message

   Test Case Derivation: Verifies IR-1 (input validation)

   How test will be performed: Automated via test_api.py::TestRecordingsRename

5.2.3 Module: Recording Database (`mRecord`)

The Recording Database module manages recording metadata, file storage, and retrieval operations.

1. test-database-allocate

   Type: Functional, Dynamic, Automatic

   Initial State: Empty temporary directory

   Input: allocate_recording() call

   Output: Recording directory created with meta.json; RecordingInfo returned with valid paths

   Test Case Derivation: Verifies FR-11 (recording allocation)

   How test will be performed: Automated via test_database.py::TestAllocateRecording

2. test-database-list-recordings

   Type: Functional, Dynamic, Automatic

   Initial State: Database with multiple recordings

   Input: list_all_recordings() call

   Output: Sorted list of RecordingInfo objects (newest first)

   Test Case Derivation: Verifies FR-12 (recording listing)

   How test will be performed: Automated via test_database.py::TestListAllRecordings

3. test-database-rename-delete

   Type: Functional, Dynamic, Automatic

   Initial State: Database with existing recording

   Input: rename_recording() and delete_recording() calls

   Output: meta.json updated with new name; directory removed on delete

   Test Case Derivation: Verifies FR-12 (recording renaming and deletion)

   How test will be performed: Automated via test_database.py::TestRenameRecording, TestDeleteRecording

4. test-database-path-traversal-protection

   Type: Functional, Dynamic, Automatic

   Initial State: Database with validation enabled

   Input: Recording IDs containing path traversal patterns (../etc/passwd)

   Output: ValueError raised; no filesystem access outside base path

   Test Case Derivation: Verifies security requirement for path validation

   How test will be performed: Automated via test_database.py::TestValidateId

5. test-database-space-usage

   Type: Functional, Dynamic, Automatic

   Initial State: Mocked disk usage

   Input: space_usage_bytes() and recording_duration_left_s() calls

   Output: Correct used/total bytes; estimated recording duration based on free space

   Test Case Derivation: Verifies FR-10 (storage status reporting)

   How test will be performed: Automated via test_database.py::TestSpaceUsage

5.2.4 Module: Gimbal Abstraction (`mGimbal`)

The Gimbal Abstraction module handles serial communication with the gimbal hardware.

1. test-gimbal-crc8

   Type: Functional, Dynamic, Automatic
   Initial State: GimbalSerial instance
   Input: Various byte sequences for CRC calculation
   Output: Correct CRC-8 values; consistent results; valid byte range (0-255)
   Test Case Derivation: Verifies communication protocol integrity
   How test will be performed: Automated via test_gimbal.py::TestCrc8Smbus

2. test-gimbal-packet-construction

   Type: Functional, Dynamic, Automatic
   Initial State: GimbalSerial with mocked serial port
   Input: create_request_data() with various request IDs and payloads
   Output: Properly formatted packets with CRC at byte 0, request ID at byte 1
   Test Case Derivation: Verifies INT-2 (gimbal protocol implementation)
   How test will be performed: Automated via test_gimbal.py::TestCreateRequestData

3. test-gimbal-angle-commands

   Type: Functional, Dynamic, Automatic
   Initial State: GimbalSerial with mocked serial port returning ACK
   Input: set_deg(tilt, pan) and get_deg() calls
   Output: True on success; correct angle values parsed from response
   Test Case Derivation: Verifies FR-2, FR-6, FR-7 (gimbal angle control)
   How test will be performed: Automated via test_gimbal.py::TestSetDeg, TestGetDeg

4. test-gimbal-led-control

   Type: Functional, Dynamic, Automatic
   Initial State: GimbalSerial with mocked serial port
   Input: set_arm_led(True/False) and set_status_led(True/False)
   Output: True on success; serial write called with correct packets
   Test Case Derivation: Verifies visual feedback control for armed status
   How test will be performed: Automated via test_gimbal.py::TestLedCommands

5. test-gimbal-info-query

   Type: Functional, Dynamic, Automatic
   Initial State: GimbalSerial with mocked response
   Input: gimbal_info() call
   Output: Tilt range, pan range, and focal length range parsed correctly
   Test Case Derivation: Verifies gimbal capability detection
   How test will be performed: Automated via test_gimbal.py::TestGimbalInfo

5.2.5 Module: Tracking (`mTrack`)

The Tracking module implements the control law for automatic rocket tracking.

1. test-tracking-queue-behavior

   Type: Functional, Dynamic, Automatic
   Initial State: Tracking instance with mocked gimbal
   Input: Sequential on_detection() calls with bounding box coordinates
   Output: Detections queued; oldest dropped when queue full; None accepted
   Test Case Derivation: Verifies FR-3, FR-7 (tracking queue management)
   How test will be performed: Automated via test_tracking.py::TestOnDetection

2. test-tracking-worker-control-law

   Type: Functional, Dynamic, Automatic
   Initial State: Tracking instance with mocked gimbal at known position
   Input: Detection coordinates off-center from frame
   Output: gimbal.set_deg() called with corrected angles based on proportional control
   Test Case Derivation: Verifies FR-7 (tracking control law)
   How test will be performed: Automated via test_tracking.py::TestWorkerControlLaw

3. test-tracking-angle-clamping

   Type: Functional, Dynamic, Automatic
   Initial State: Tracking instance with mocked gimbal near angle limits
   Input: Detection that would push angles beyond valid range
   Output: New angles clamped to valid tilt/pan ranges
   Test Case Derivation: Verifies IR-1 (tracking command validation)
   How test will be performed: Automated via test_tracking.py::TestWorkerControlLaw

4. test-tracking-stop

   Type: Functional, Dynamic, Automatic

   Initial State: Running Tracking instance

   Input: stop() call

   Output: Stop event set; worker thread joined successfully

   Test Case Derivation: Verifies clean shutdown of tracking thread

   How test will be performed: Automated via test_tracking.py::TestStop

5.2.6 Module: CV Process Management (`mCVManagement`)

The CV Process Management module handles the lifecycle and IPC with the CV worker process.

1. test-cv-management-osd-send

   Type: Functional, Dynamic, Automatic

   Initial State: CVProcessManagement with open IPC connection

   Input: send_osd_data() with OSDData

   Output: Data sent over IPC; exceptions swallowed safely

   Test Case Derivation: Verifies FR-10 (OSD data transmission to CV process)

   How test will be performed: Automated via test_cv_process_mgmt.py::TestSendOsdData

2. test-cv-management-recording-commands

   Type: Functional, Dynamic, Automatic

   Initial State: CVProcessManagement with open IPC connection

   Input: start_recording() with RecordingInfo; stop_recording()

   Output: Recording info sent; StopRecording sentinel sent

   Test Case Derivation: Verifies FR-11 (recording command relay to CV process)

   How test will be performed: Automated via test_cv_process_mgmt.py::TestRecordingCommands

3. test-cv-management-pause-resume

   Type: Functional, Dynamic, Automatic

   Initial State: CVProcessManagement with active process

   Input: pause_pipeline() and resume_pipeline() calls

   Output: Process terminated on pause; resume event set on resume

Test Case Derivation: Verifies pipeline control for download operations

How test will be performed: Automated via test_cv_process_mgmt.py::TestPauseResume

4. test-cv-management-recv-loop

   Type: Functional, Dynamic, Automatic

   Initial State: CVProcessManagement with mocked connection

   Input: Simulated CVData and PreviewData via recv loop

   Output: Callbacks dispatched correctly; connection closed on EOF

   Test Case Derivation: Verifies data flow from CV process to control process

   How test will be performed: Automated via test_cv_process_mgmt.py::TestRecvLoop

5.2.7 Module: Live Video Process Management (`mVideoManagement`)

The Live Video Process Management module supervises the live video output process.

1. test-livestream-construction

   Type: Functional, Dynamic, Automatic

   Initial State: System ready for process management

   Input: LivestreamProcessManagement instantiation

   Output: Daemon thread started for process supervision

   Test Case Derivation: Verifies FR-8, FR-9 (live video process lifecycle)

   How test will be performed: Automated via test_livestream_mgmt.py

2. test-livestream-process-loop

   Type: Functional, Dynamic, Automatic

   Initial State: LivestreamProcessManagement instance

   Input: Simulated process loop iteration

   Output: subprocess.Popen called with livestream command; process monitored

   Test Case Derivation: Verifies live video process supervision

   How test will be performed: Automated via test_livestream_mgmt.py

5.2.8 Module: Transcode API (mAPI)

The Transcode API provides endpoints for stabilized video preview and download.

1. test-transcode-routes-404

   Type: Functional, Dynamic, Automatic
   Initial State: Flask app with transcode routes registered; recording not found
   Input: GET /api/recordings/¡id¿/preview-stabilized and /download-stabilized
   Output: HTTP 404 when recording not found
   Test Case Derivation: Verifies FR-12 (proper error handling for missing recordings)
   How test will be performed: Automated via test_transcode_api.py::TestTranscodeRoutes

2. test-transcode-streaming

   Type: Functional, Dynamic, Automatic
   Initial State: Flask app with valid recording
   Input: GET preview-stabilized or download-stabilized with valid recording ID
   Output: HTTP 200 with video/webm content type; on_download_start/end called
   Test Case Derivation: Verifies FR-12 (stabilized video streaming)
   How test will be performed: Automated via test_transcode_api.py::TestTranscodeRoutes

3. test-transcode-pipe-cleanup

   Type: Functional, Dynamic, Automatic
   Initial State: Named pipe exists on filesystem
   Input: _cleanup_pipe() call
   Output: Pipe file removed; no error if pipe missing; OSError swallowed
   Test Case Derivation: Verifies resource cleanup for transcode operations
   How test will be performed: Automated via test_transcode_api.py::TestCleanupPipe

5.2.9 Module: Transcode Process (mTranscode)

The Transcode Process module handles offline video transcoding with stabilization.

1. test-transcode-read-log

   Type: Functional, Dynamic, Automatic
   Initial State: TranscodeProcess instance with mocked GStreamer

Input: JSONL log file with OSD entries

Output: List of OSDData objects parsed correctly; invalid lines skipped

Test Case Derivation: Verifies FR-12 (log parsing for overlay generation)

How test will be performed: Automated via test_transcode_main.py::TestTranscodeProcessReadLog

2. test-transcode-shader-probe

   Type: Functional, Dynamic, Automatic

   Initial State: TranscodeProcess with OSD data list

   Input: Shader probe calls with various PTS values

   Output: Correct OSD data selected; pointer advanced appropriately for preview vs download modes

   Test Case Derivation: Verifies overlay synchronization with video frames

   How test will be performed: Automated via test_transcode_main.py::TestTranscodeProcessShaderPro

3. test-transcode-bus-call

   Type: Functional, Dynamic, Automatic

   Initial State: TranscodeProcess with mocked GStreamer bus

   Input: EOS, WARNING, and ERROR messages

   Output: Loop quit on EOS and ERROR; continued on WARNING

   Test Case Derivation: Verifies proper GStreamer pipeline event handling

   How test will be performed: Automated via test_transcode_main.py::TestTranscodeProcessBusCall

5.2.10 Module: CV Process (`mCV`)

The CV Process module handles video acquisition, object detection, and recording.

1. test-cv-format-time

   Type: Functional, Dynamic, Automatic

   Initial State: CV process utilities loaded

   Input: Millisecond timestamps

   Output: Human-readable formatted time strings with milliseconds

   Test Case Derivation: Verifies utility function for timestamp formatting in overlays

   How test will be performed: Automated via test_cv_main.py::TestFormatTime

2. test-cv-update-osd

   Type: Functional, Dynamic, Automatic
   Initial State: Mocked GStreamer osd and shader elements
   Input: OSDData with gimbal angles, GPS coordinates, IP addresses
   Output: Text property set with formatted OSD; shader uniforms updated
   Test Case Derivation: Verifies FR-8, FR-9, FR-10 (OSD overlay generation)
   How test will be performed: Automated via test_cv_main.py::TestUpdateOsd

5.2.11 Module: Common IPC (Shared Modules)

The Common IPC module provides data structures and communication utilities.

1. test-bounding-box-center

   Type: Functional, Dynamic, Automatic
   Initial State: BoundingBox instance
   Input: Various bounding box coordinates
   Output: Correct center point (cx, cy) calculated
   Test Case Derivation: Verifies geometry calculations for tracking
   How test will be performed: Automated via test_ipc.py::TestBoundingBoxCenter

2. test-bounding-box-rotate-90

   Type: Functional, Dynamic, Automatic
   Initial State: BoundingBox instance
   Input: Bounding box for 90-degree rotated frame
   Output: Correctly transformed bounding box
   Test Case Derivation: Verifies coordinate transformation for rotated camera
   How test will be performed: Automated via test_ipc.py::TestBoundingBoxRotate90

3. test-dataclass-construction

   Type: Functional, Dynamic, Automatic
   Initial State: IPC module loaded
   Input: Construction of CVData, OSDData, PreviewData, RecordingInfo, StopRecording
   Output: Objects created with correct field values
   Test Case Derivation: Verifies data structure integrity
   How test will be performed: Automated via test_ipc.py::TestDataclasses

5.2.12 Module: Common IPC Buffer

The IPC Buffer module provides shared memory circular buffer for high-throughput video data.

1. test-ipc-buffer-sender

   Type: Functional, Dynamic, Automatic
   Initial State: IPCBufferSender with mocked shared memory
   Input: send() calls with fixed-size payloads
   Output: Head advanced; data written at correct offset; returns False when full
   Test Case Derivation: Verifies efficient video frame buffering
   How test will be performed: Automated via test_ipc_buffer.py::TestIPCBufferSender

2. test-ipc-buffer-receiver

   Type: Functional, Dynamic, Automatic
   Initial State: IPCBufferReceiver with mocked shared memory containing data
   Input: receive() calls with various head/tail positions
   Output: Data retrieved correctly; tail advanced; overrun handled gracefully
   Test Case Derivation: Verifies reliable video frame retrieval
   How test will be performed: Automated via test_ipc_buffer.py::TestIPCBufferReceiver

5.2.13 Module: Common Watchdog

The Watchdog module provides timeout-based process monitoring.

1. test-watchdog-refresh

   Type: Functional, Dynamic, Automatic
   Initial State: Watchdog instance with timeout and callback
   Input: refresh() calls
   Output: Timer created and started; previous timer cancelled
   Test Case Derivation: Verifies RFR-2 (watchdog functionality for process monitoring)
   How test will be performed: Automated via test_watchdog.py::TestWatchdogRefresh

2. test-watchdog-clear

   Type: Functional, Dynamic, Automatic
   Initial State: Watchdog with active timer

Input: clear() call

Output: Timer cancelled; reference cleared; idempotent when no timer

Test Case Derivation: Verifies safe watchdog shutdown

How test will be performed: Automated via test_watchdog.py::TestWatchdogClear

5.2.14 Module: Preview (`mPreview`)

The Preview module displays live video feed in the UI.

1. test-control-page-render

   Type: Functional, Dynamic, Automatic

   Initial State: React component with mocked dependencies

   Input: Various status states (null, with/without preview, armed/disarmed, recording)

   Output: Preview image rendered when available; spinner shown when loading; REC/ARMED indicators displayed correctly

   Test Case Derivation: Verifies FR-10 (real-time preview display)

   How test will be performed: Automated via test_pages/control.test.tsx

2. test-control-page-bbox-overlay

   Type: Functional, Dynamic, Automatic

   Initial State: Control page with status containing bbox

   Input: Status with bounding box coordinates and confidence

   Output: Bounding box overlay rendered with confidence value

   Test Case Derivation: Verifies FR-3 (detection visualization)

   How test will be performed: Automated via test_pages/control.test.tsx

5.2.15 Module: Manual Control (`mManual`)

The Manual Control module provides UI for gimbal control.

1. test-controls-card-render

   Type: Functional, Dynamic, Automatic

   Initial State: ControlsCard component with mocked API

   Input: Various system states (armed/disarmed)

   Output: Directional controls rendered; manual move API called on interaction

   Test Case Derivation: Verifies FR-2, FR-6 (manual gimbal control UI)

   How test will be performed: Automated via test_components/ControlsCard.test.tsx

2. test-control-page-drag

   Type: Functional, Dynamic, Automatic

   Initial State: Control page in disarmed mode

   Input: Pointer drag gestures on preview area

   Output: manualMoveTo API called with calculated tilt/pan values

   Test Case Derivation: Verifies FR-2 (intuitive manual control)

   How test will be performed: Automated via test_pages/control.test.tsx

5.2.16 Module: Recording Management (`mRecordMng`)

The Recording Management module provides UI for listing, renaming, deleting, and previewing recordings.

1. test-recordings-page-loading

   Type: Functional, Dynamic, Automatic

   Initial State: Recordings page mounted

   Input: API loading state

   Output: Loading spinner displayed

   Test Case Derivation: Verifies UI feedback during data fetch

   How test will be performed: Automated via test_pages/recordings.test.tsx

2. test-recordings-page-empty

   Type: Functional, Dynamic, Automatic

   Initial State: Recordings page with empty list

   Input: API returns empty recordings array

   Output: "No recordings" message displayed

   Test Case Derivation: Verifies FR-12 (empty state handling)

   How test will be performed: Automated via test_pages/recordings.test.tsx

3. test-recordings-page-list

   Type: Functional, Dynamic, Automatic

   Initial State: Recordings page with sample data

   Input: API returns list of recordings

   Output: Recording names rendered; dates and durations formatted

Test Case Derivation: Verifies FR-12 (recording listing)

How test will be performed: Automated via test_pages/recordings.test.tsx

4. test-recordings-delete

   Type: Functional, Dynamic, Automatic

   Initial State: Recordings page with sample recording

   Input: Delete button clicked with confirmation

   Output: deleteRecording API called with correct ID

   Test Case Derivation: Verifies FR-12 (recording deletion with confirmation)

   How test will be performed: Automated via test_pages/recordings.test.tsx

5. test-recordings-rename

   Type: Functional, Dynamic, Automatic

   Initial State: Recordings page with editable name

   Input: Name changed and blurred (or Escape pressed)

   Output: renameRecording API called with new name; cancelled on Escape

   Test Case Derivation: Verifies FR-12 (recording renaming)

   How test will be performed: Automated via test_pages/recordings.test.tsx

5.2.17 Module: Configuration (`mConfig`)

The Configuration module provides UI for system settings.

1. test-language-selector

   Type: Functional, Dynamic, Automatic

   Initial State: LanguageSelector component

   Input: Language selection change

   Output: Language atom updated; i18n activated

   Test Case Derivation: Verifies PI-2, EZ-4 (language configuration)

   How test will be performed: Automated via test_components/LanguageSelector.test.tsx

2. test-configuration-menu

   Type: Functional, Dynamic, Automatic

   Initial State: ConfigurationMenu component

   Input: Settings changes (temperature unit, drag sensitivity, invert drag)

Output: Setting atoms updated with new values

Test Case Derivation: Verifies UPR-1, UPR-3 (user preference configuration)

How test will be performed: Automated via test_components/ConfigurationMenu.test.tsx

5.2.18 Module: API Client (Network Layer)

The API Client module provides HTTP communication with the backend.

1. test-api-client-urls

   Type: Functional, Dynamic, Automatic

   Initial State: ApiClient instance

   Input: Base URL configuration

   Output: Correct endpoint URLs generated for status stream, recordings, previews

   Test Case Derivation: Verifies FR-10, FR-12 (API endpoint construction)

   How test will be performed: Automated via test_network/api.test.ts

2. test-api-client-automatic-discovery

   Type: Functional, Dynamic, Automatic

   Initial State: Network with multiple potential server URLs

   Input: createAutomatic() with probe URLs

   Output: Client created with first responding URL; throws if all fail

   Test Case Derivation: Verifies SCR-3 (automatic server discovery)

   How test will be performed: Automated via test_network/api.test.ts

3. test-api-client-methods

   Type: Functional, Dynamic, Automatic

   Initial State: ApiClient with mocked fetch

   Input: manualMove(), arm(), disarm(), startRecording(), stopRecording(), listRecordings(), renameRecording(), deleteRecording()

   Output: Correct HTTP methods and URLs; proper request bodies; ApiError thrown on failure

   Test Case Derivation: Verifies FR-2, FR-6, FR-10, FR-11, FR-12 (API operations)

   How test will be performed: Automated via test_network/api.test.ts

5.2.19 Module: Rocam Provider (React Context)

The Rocam Provider module manages API client instance and global state for React components.

1. test-rocam-provider-initialization

   Type: Functional, Dynamic, Automatic
   Initial State: RocamProvider component with mocked createAutomatic
   Input: Provider mounted with children
   Output: API client created; status polling started; loading state managed
   Test Case Derivation: Verifies FR-10 (provider initializes API connection on mount)
   How test will be performed: Automated via test_network/rocamProvider.test.tsx

2. test-rocam-provider-error-handling

   Type: Nonfunctional, Robustness, Automatic
   Initial State: RocamProvider with mocked API that fails to connect
   Input: API initialization failure
   Output: Error state set; error message displayed; component doesn't crash
   Test Case Derivation: Verifies RFR-1 (graceful handling of connection failures)
   How test will be performed: Automated via test_network/rocamProvider.test.tsx

3. test-rocam-provider-polling

   Type: Functional, Dynamic, Automatic
   Initial State: RocamProvider with mocked API client
   Input: Status stream events and polling intervals
   Output: Status state updated; polling continues with proper cleanup on unmount
   Test Case Derivation: Verifies FR-10 (real-time status updates via SSE)
   How test will be performed: Automated via test_network/rocamProvider.test.tsx

4. test-use-rocam-hook

   Type: Functional, Dynamic, Automatic
   Initial State: Component wrapped in RocamProvider
   Input: useRocam() hook called
   Output: API client and status accessible to consuming components
   Test Case Derivation: Verifies React context API exposes required values
   How test will be performed: Automated via test_network/rocamProvider.test.tsx

5.2.20 Module: Utility Formatters

The Utility Formatters module provides data formatting functions.

1. test-formatters-degrees

   Type: Functional, Dynamic, Automatic
   Initial State: Formatter functions loaded
   Input: Various numeric values for degrees, FPS, percentages
   Output: Correctly formatted strings with appropriate precision and units
   Test Case Derivation: Verifies PI-1 (intuitive value display)
   How test will be performed: Automated via test_utils/formatters.test.ts

2. test-formatters-temperature

   Type: Functional, Dynamic, Automatic
   Initial State: Formatter functions loaded
   Input: Celsius temperatures; Fahrenheit unit preference
   Output: Correct conversion and formatting with degree symbol
   Test Case Derivation: Verifies temperature display in both units
   How test will be performed: Automated via test_utils/formatters.test.ts

3. test-formatters-bytes

   Type: Functional, Dynamic, Automatic
   Initial State: Formatter functions loaded
   Input: Byte counts of various magnitudes
   Output: Human-readable sizes (B, KB, MB, GB)
   Test Case Derivation: Verifies FR-10 (storage display)
   How test will be performed: Automated via test_utils/formatters.test.ts

4. test-formatters-duration

   Type: Functional, Dynamic, Automatic
   Initial State: Formatter functions loaded
   Input: Millisecond durations
   Output: Formatted HH:MM:SS or relative time strings
   Test Case Derivation: Verifies FR-12 (recording duration display)
   How test will be performed: Automated via test_utils/formatters.test.ts

5.2.21 Module: Internationalization (i18n)

The Internationalization module provides multi-language support.

1. test-i18n-activation

   Type: Functional, Dynamic, Automatic

   Initial State: i18n module with mocked lingui

   Input: dynamicActivate() with locale string

   Output: Messages loaded; i18n activated with correct locale

   Test Case Derivation: Verifies PI-2 (language switching)

   How test will be performed: Automated via test_i18n.test.ts

5.2.22 Module: Application Routing (App)

The App module handles page routing and navigation.

1. test-app-routing

   Type: Functional, Dynamic, Automatic

   Initial State: App component with MemoryRouter

   Input: Navigation to / and /recordings routes

   Output: Correct page components rendered; unknown paths redirect to /

   Test Case Derivation: Verifies EZ-1 (navigation between pages)

   How test will be performed: Automated via test_App.test.tsx

5.3 Tests for Nonfunctional Requirements

Unit tests for nonfunctional requirements verify performance characteristics, reliability, and quality attributes of individual modules. These tests focus on measurable behaviors such as timing, resource usage, error handling, and robustness.

5.3.1 Module: State Management (`mState`)

1. test-state-status-performance

   Type: Nonfunctional, Performance, Automatic

   Initial State: StateManagement with mocked dependencies and cached metrics

   Input: status() called repeatedly

   Output: Status returned within milliseconds; cached values returned without recalculation

Test Case Derivation: Verifies SLR-2, SLR-3 (status query performance for real-time monitoring)

How test will be performed: Automated via test_state_management_class.py::TestStatus

2. test-state-error-handling

   Type: Nonfunctional, Robustness, Automatic

   Initial State: StateManagement with mocked components that may raise exceptions

   Input: Various operations with components configured to raise exceptions

   Output: Operations fail gracefully; no unhandled exceptions propagate

   Test Case Derivation: Verifies RFR-1 (error handling for robustness)

   How test will be performed: Automated via test_state_management_class.py

5.3.2 Module: API Gateway (`mAPI`)

1. test-api-error-responses

   Type: Nonfunctional, Robustness, Automatic

   Initial State: Flask test client

   Input: Invalid JSON, missing required fields, malformed requests

   Output: HTTP 400 with descriptive error messages; no server crashes

   Test Case Derivation: Verifies RFR-1 (API error handling and feedback)

   How test will be performed: Automated via test_api.py::TestRecordingsRename

5.3.3 Module: SSE (Server-Sent Events)

1. test-sse-broadcast-performance

   Type: Nonfunctional, Performance, Automatic

   Initial State: MessageAnnouncer with multiple listeners registered

   Input: Rapid announce() calls with status updates

   Output: All listeners receive messages; full queues drop messages silently (no blocking)

   Test Case Derivation: Verifies SLR-3, RFR-4 (real-time status streaming without back-pressure)

   How test will be performed: Automated via test_sse.py::TestMessageAnnouncer

2. test-sse-thread-safety

   Type: Nonfunctional, Robustness, Automatic

Initial State: MessageAnnouncer with concurrent operations

Input: Simultaneous listen(), remove(), and announce() from multiple threads

Output: No race conditions; all operations complete safely

Test Case Derivation: Verifies thread safety for concurrent UI connections

How test will be performed: Automated via test_sse.py::TestMessageAnnouncer

5.3.4 Module: IPC Buffer

1. test-ipc-buffer-overrun-handling

   Type: Nonfunctional, Robustness, Automatic

   Initial State: IPCBufferReceiver with head significantly ahead of tail (overrun condition)

   Input: receive() call during overrun

   Output: Tail advanced to catch up; data retrieved without corruption

   Test Case Derivation: Verifies RFR-4 (graceful handling of producer/consumer rate mismatch)

   How test will be performed: Automated via test_ipc_buffer.py::TestIPCBufferReceiver

2. test-ipc-buffer-memory-safety

   Type: Nonfunctional, Robustness, Automatic

   Initial State: IPCBuffer with various head/tail positions including edge cases

   Input: Operations at buffer boundaries

   Output: No out-of-bounds access; proper ring buffer wrapping

   Test Case Derivation: Verifies memory safety for shared video data

   How test will be performed: Automated via test_ipc_buffer.py

5.3.5 Module: CV Process Management (`mCVManagement`)

1. test-cv-management-exception-safety

   Type: Nonfunctional, Robustness, Automatic

   Initial State: CVProcessManagement with connection in various states

   Input: Operations during broken pipe, EOF, and other error conditions

   Output: Exceptions caught and swallowed; component continues operating

   Test Case Derivation: Verifies RFR-2 (fault tolerance for process communication)

   How test will be performed: Automated via test_cv_process_mgmt.py::TestExceptionHandling

5.3.6 Module: Recording Database (`mRecord`)

1. test-database-error-handling

   Type: Nonfunctional, Robustness, Automatic
   Initial State: Database with simulated filesystem errors
   Input: Operations with mocked os/path functions raising OSError
   Output: Graceful degradation; sensible defaults returned; exceptions handled
   Test Case Derivation: Verifies RFR-1, CR-1 (error handling and data integrity)
   How test will be performed: Automated via test_database.py::TestGetSizeBytes

5.3.7 Module: Watchdog

1. test-watchdog-timeout-accuracy

   Type: Nonfunctional, Performance, Automatic
   Initial State: Watchdog with specific timeout value
   Input: refresh() calls at various intervals
   Output: Timer created with correct timeout; callback triggered after timeout
   Test Case Derivation: Verifies PAR-1 (timing accuracy for process monitoring)
   How test will be performed: Automated via test_watchdog.py

5.3.8 Module: System Status (`mStatus`)

1. test-system-status-caching

   Type: Nonfunctional, Performance, Automatic
   Initial State: SystemStatusMonitor with mocked jtop
   Input: Multiple getter calls between updates
   Output: Cached values returned immediately; no redundant hardware queries
   Test Case Derivation: Verifies SLR-2, SLR-3 (efficient status monitoring)
   How test will be performed: Automated via test_system_status.py

2. test-system-status-callback-updates

   Type: Nonfunctional, Correctness, Automatic
   Initial State: SystemStatusMonitor with initial readings
   Input: jtop callback with updated sensor values
   Output: All cached metrics updated to new values

Test Case Derivation: Verifies EZ-3 (accurate system telemetry)

How test will be performed: Automated via test_system_status.py::TestSystemStatusMonitor

5.3.9 Module: Common Utils

The Utils module provides network and scheduling utilities.

1. test-ip4-addresses

   Type: Functional, Dynamic, Automatic

   Initial State: Utils module with mocked netifaces

   Input: Network interface configurations with various IPv4 addresses

   Output: List of IP addresses extracted correctly; empty list when no interfaces

   Test Case Derivation: Verifies FR-10 (device IP address reporting for UI)

   How test will be performed: Automated via test_utils.py::TestIp4Addresses

2. test-scheduler-validation

   Type: Nonfunctional, Robustness, Automatic

   Initial State: Scheduler functions with mocked OS calls

   Input: Various priority values (valid and invalid) for FIFO, OTHER, and BATCH schedulers

   Output: ValueError raised for out-of-range priorities; successful calls for valid values

   Test Case Derivation: Verifies PAR-1 (process scheduling priority validation)

   How test will be performed: Automated via test_utils.py::TestSchedulerValidation

5.3.10 Module: Recording-Only State Management

The Recording-Only State Management module provides a limited state management implementation for recording management mode.

1. test-state-management-recording-only-noops

   Type: Functional, Dynamic, Automatic

   Initial State: StateManagementRecordingManagementOnly instance

   Input: Calls to arm(), disarm(), manual_move(), manual_move_to(), start_recording(), stop_recording()

   Output: All operations are no-ops (do not raise exceptions)

   Test Case Derivation: Verifies that recording-only mode safely ignores control operations

   How test will be performed: Automated via test_state_management_recording_only.py::TestStateMana

2. test-state-management-recording-only-status

   Type: Functional, Dynamic, Automatic

   Initial State: StateManagementRecordingManagementOnly with mocked database and system status

   Input: status() call

   Output: StatusResponse with is_recording=False, armed=False, system metrics populated

   Test Case Derivation: Verifies FR-10 (status reporting in recording-only mode)

   How test will be performed: Automated via test_state_management_recording_only.py::TestStateMana

3. test-control-process-main-entry

   Type: Functional, Dynamic, Automatic

   Initial State: Mocked StateManagement and threading

   Input: run_control_process() and run_recording_management() calls

   Output: API thread started and joined; StateManagement instantiated

   Test Case Derivation: Verifies control process entry points initialize correctly

   How test will be performed: Automated via test_state_management_recording_only.py::TestControlPr

5.3.11 Module: Main Entry Point

The Main Entry Point module handles command-line dispatch to different process modes.

1. test-main-dispatch-control

   Type: Functional, Dynamic, Automatic

   Initial State: Mocked sys.argv with no arguments

   Input: Default execution (no command line args)

   Output: run_control_process() called

   Test Case Derivation: Verifies default mode launches control process

   How test will be performed: Automated via test_main_entry.py::TestMainDispatch

2. test-main-dispatch-cv

   Type: Functional, Dynamic, Automatic

   Initial State: Mocked sys.argv with ["cv"] argument

   Input: CV process launch command

Output: run_cv_process() called

Test Case Derivation: Verifies FR-3 (CV process launch)

How test will be performed: Automated via test_main_entry.py::TestMainDispatch

3. test-main-dispatch-livestream

   Type: Functional, Dynamic, Automatic

   Initial State: Mocked sys.argv with ["livestream"] argument

   Input: Livestream process launch command

   Output: run_livestream_process() called

   Test Case Derivation: Verifies FR-8, FR-9 (live video process launch)

   How test will be performed: Automated via test_main_entry.py::TestMainDispatch

4. test-main-dispatch-transcode

   Type: Functional, Dynamic, Automatic

   Initial State: Mocked sys.argv with ["download-stabilized"] or ["preview-stabilized"] arguments

   Input: Transcode process launch command with video and log paths

   Output: start_transcode_process() called with correct mode and paths

   Test Case Derivation: Verifies FR-12 (transcode process launch for recording stabilization)

   How test will be performed: Automated via test_main_entry.py::TestMainDispatch

5. test-main-dispatch-unknown

   Type: Nonfunctional, Robustness, Automatic

   Initial State: Mocked sys.argv with unknown command

   Input: Unknown command line argument

   Output: System exits with code 1

   Test Case Derivation: Verifies proper error handling for invalid commands

   How test will be performed: Automated via test_main_entry.py::TestMainDispatch

5.3.12 Module: Utility Formatters

1. test-formatters-edge-cases

   Type: Nonfunctional, Robustness, Automatic

   Initial State: Formatter functions loaded

Input: Edge values (null, undefined, negative, zero, very large numbers)
Output: Sensible defaults; no crashes; formatted output for all inputs
Test Case Derivation: Verifies RFR-1 (robustness against unexpected data)
How test will be performed: Automated via test_utils/formatters.test.ts

2. test-formatters-internationalization

Type: Nonfunctional, Internationalization, Automatic
Initial State: Formatters with various locale considerations
Input: Values requiring locale-specific formatting
Output: Consistent formatting regardless of system locale
Test Case Derivation: Verifies PI-2 (consistent international display)
How test will be performed: Automated via test_utils/formatters.test.ts

5.3.13 Module: UI Components

1. test-ui-loading-states

Type: Nonfunctional, Usability, Automatic
Initial State: Components with loading/null states
Input: Null or undefined data
Output: Loading spinners or placeholder content; no crashes
Test Case Derivation: Verifies EZ-2 (immediate feedback for pending operations)
How test will be performed: Automated via component test files (e.g., test_pages/control.test.tsx)

2. test-ui-error-handling

Type: Nonfunctional, Robustness, Automatic
Initial State: Components with mocked API that returns errors
Input: API errors, network failures, malformed responses
Output: Graceful error display; component remains interactive
Test Case Derivation: Verifies RFR-1 (UI error resilience)
How test will be performed: Automated via component test files

5.4 Traceability Between Test Cases and Modules

The following tables provide traceability between unit test cases and the modules defined in the Module Guide (MG). Each test case verifies specific functionality of its corresponding module, ensuring that all implemented leaf modules have adequate test coverage.

Table 5: Traceability Between Unit Test Cases and Back-end Modules

Module	**Unit Test Cases**
`mState` (State Management)	test-state-management-arm-disarm; test-state-management-manual-move; test-state-management-manual-move-clamped; test-state-management-recording; test-state-management-download-pause; test-state-status-performance; test-state-error-handling
`mAPI` (API Gateway)	test-api-endpoints-exist; test-api-recordings-crud; test-api-validation; test-api-error-responses
`mRecord` (Recording Database)	test-database-allocate; test-database-list-recordings; test-database-rename-delete; test-database-path-traversal-protection; test-database-space-usage; test-database-error-handling
`mGimbal` (Gimbal Abstraction)	test-gimbal-crc8; test-gimbal-packet-construction; test-gimbal-angle-commands; test-gimbal-led-control; test-gimbal-info-query
`mTrack` (Tracking)	test-tracking-queue-behavior; test-tracking-worker-control-law; test-tracking-angle-clamping; test-tracking-stop
`mCVManagement` (CV Process Management)	test-cv-management-osd-send; test-cv-management-recording-commands; test-cv-management-pause-resume; test-cv-management-recv-loop; test-cv-management-exception-safety
`mVideoManagement` (Live Video Management)	test-livestream-construction; test-livestream-process-loop
`mTranscodeManagement` (Transcode Management)	test-livestream-construction; test-livestream-process-loop
`mTranscode` (Transcode Process)	test-transcode-read-log; test-transcode-shader-probe; test-transcode-bus-call
`mCV` (CV Process)	test-cv-format-time; test-cv-update-osd
`mVideo` (Live Video Process)	test-livestream-construction; test-livestream-process-loop

Continued on next page

Table 5 – Continued from previous page

Module	**Unit Test Cases**
Common IPC	test-bounding-box-center; test-bounding-box-rotate-90; test-dataclass-construction
Common IPC Buffer	test-ipc-buffer-sender; test-ipc-buffer-receiver; test-ipc-buffer-overrun-handling; test-ipc-buffer-memory-safety
Common Watchdog	test-watchdog-refresh; test-watchdog-clear; test-watchdog-timeout-accuracy
Common System Status	test-system-status-caching; test-system-status-callback-updates
Common Utils	test-ip4-addresses; test-scheduler-validation
Recording-Only State Management	test-state-management-recording-only-noops; test-state-management-recording-only-status; test-control-process-main-entry
Main Entry Point	test-main-dispatch-control; test-main-dispatch-cv; test-main-dispatch-livestream; test-main-dispatch-transcode; test-main-dispatch-unknown
Transcode API (`mAPI`)	test-transcode-routes-404; test-transcode-streaming; test-transcode-pipe-cleanup
SSE (`mAPI`)	test-sse-broadcast-performance; test-sse-thread-safety

Table 6: Traceability Between Unit Test Cases and Frontend Modules

Module	**Unit Test Cases**
`mPreview` (Preview)	test-control-page-render; test-control-page-bbox-overlay
`mManual` (Manual Control)	test-controls-card-render; test-control-page-drag
`mRecordMng` (Recording Management)	test-recordings-page-loading; test-recordings-page-empty; test-recordings-page-list; test-recordings-delete; test-recordings-rename
`mConfig` (Configuration)	test-language-selector; test-configuration-menu
API Client	test-api-client-urls; test-api-client-automatic-discovery; test-api-client-methods
Rocam Provider	test-rocam-provider-initialization; test-rocam-provider-error-handling; test-rocam-provider-polling; test-use-rocam-hook
Utility Formatters	test-formatters-degrees; test-formatters-temperature; test-formatters-bytes; test-formatters-duration; test-formatters-edge-cases; test-formatters-internationalization
Internationalization	test-i18n-activation
App (Routing)	test-app-routing
UI Components (Individual)	test-system-status-card; test-navbar; test-controls-card; test-language-selector; test-configuration-menu; test-ui-loading-states; test-ui-error-handling

Note: The following modules are listed in MG but are not verified by unit tests (as documented in the Unit Testing Scope section):

- `mStream` (Video Stream Abstraction) — Third-party library (Nvidia Deepstream SDK)
- `mSerial` (Serial Abstraction) — Third-party library (pyserial)
- `mStatus` (System Status Module) — Hardware-dependent library (jetson-stats)

Container modules (`mJetson`, `mControl`, `mUI`) are also not directly tested as they are organizational constructs; their leaf modules are tested individually.

References

Vision-Guided Tracker Team. Software architecture (module guide), 2025a. Project document, this repository.

Vision-Guided Tracker Team. Software detailed design (module interface specification), 2025b. Project document, this repository.

Vision-Guided Tracker Team. Software requirements specification, 2025c. Project document, this repository.

6 Appendix

6.1 User Survey Questions

Please answer each question based on your experience during the field test. For questions rated on a scale, circle the number that best reflects your opinion: 1 = Strongly Disagree, 2 = Disagree, 3 = Neutral, 4 = Agree, 5 = Strongly Agree.

User Interface

1. The interface was readable and easy to see outdoors. (1 2 3 4 5)
2. The interface looked professional and well-designed. (1 2 3 4 5)
3. The important information was immediately visible on the screen. (1 2 3 4 5)
4. No unnecessary or confusing information was shown. (1 2 3 4 5)
5. The system was easy to learn and operate after a short time. (1 2 3 4 5)
6. The terms and labels used in the interface were clear and understandable. (1 2 3 4 5)

System Performance

1. The tracking footage was stable and smooth. (1 2 3 4 5)
2. The tracking system maintained lock on the rocket reliably. (1 2 3 4 5)
3. The system operated as expected without major errors. (1 2 3 4 5)
4. Once armed, the system operated fully autonomously. (1 2 3 4 5)
5. I was able to fully operate and monitor the system remotely. (1 2 3 4 5)

User Experience and Feedback

1. Were any colors, symbols, or content elements disrespectful or inappropriate to you? *(Please describe if applicable)*

   _____

   _____

2. What improvements would you suggest for future versions of RoCam?

   _____

   _____

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

Jianqing Liu

1. **What went well:** I was responsible for the Part 4 of this deliverable, focusing on all the system tests. I think it was really easy to come up with all the test cases because We already completed the SRS.

2. **Pain points and resolution:** I think some of the test cases are very verbose. For some of the tests, how test will be performed is just the same as the input and output condition of the test case.

Mike Chen

1. **What went well:** I contributed primarily to Part 3 of this deliverable, focusing on the verification and validation plan and the automation tools section. The structure

and clarity of this part improved substantially after integrating team and stakeholder feedback. I ensured that each verification activity was directly tied to the SRS and our test framework.

2. **Pain points and resolution:** I initially faced several issues with GitHub CI configuration—some commits overwrote teammates' work and caused LaTeX compilation failures. I resolved these by restoring the affected commits, redesigning the YAML workflows, and setting up branch protection rules. Revisions based on feedback also required extensive reformatting to achieve consistency across sections.

3. **Knowledge and skills to acquire:** I plan to improve my skills in CI/CD automation and verification planning, particularly integrating automated builds, test execution, and documentation generation through GitHub Actions.

4. **Approaches to acquire skills:** I will (1) study advanced GitHub Actions pipelines from open-source projects and (2) create a smaller test repository to simulate multi-stage verification workflows. I selected the second approach because it provides hands-on experience with merge control, build automation, and verification feedback integration.

Frank

1. **What went well:** I contributed to both Part 2 and Part 3 of the deliverable, ensuring that the design and verification sections were fully aligned with the SRS. My work focused on refining the test structure, verifying requirement traceability, and improving the flow between verification items and their corresponding system requirements.

2. **Pain points and resolution:** The main challenge was maintaining consistent terminology and depth of explanation between sections written by different team members. To resolve this, I reviewed the SRS and cross-checked every requirement reference to ensure correctness and coherence across parts.

3. **Knowledge and skills to acquire:** I plan to strengthen my knowledge of requirements traceability and automated testing frameworks to improve coverage and maintain consistency with system goals.

4. **Approaches to acquire skills:** I will (1) review documentation on structured traceability mapping and (2) develop example UI and integration tests that link directly to specific SRS requirements. I chose the second approach because applying traceability in practice will reinforce both documentation and test alignment.

Xiaotian Lou

1. **What went well:** I focused mainly on Part 2 of the deliverable, organizing the SRS and design verification sections. The verification checklists and phase structure improved document readability and made the review process more systematic.

2. **Pain points and resolution:** The primary difficulty was harmonizing writing styles and section formatting across contributors. I standardized tables, checklists, and layout conventions to produce a consistent and professional presentation.

3. **Knowledge and skills to acquire:** I aim to gain more experience in formal verification documentation and structured validation reporting to improve the traceability and completeness of future deliverables.

4. **Approaches to acquire skills:** I will (1) study professional verification and validation templates from engineering projects and (2) apply checklist-based verification methods in upcoming development stages. I selected the first approach to strengthen my understanding of documentation standards and best practices.